



Lab 2

Writing a Shell Script for a Simple Antivirus Daemon

Objectives

- To practice writing shell scripts that run as long-lived monitoring daemons
- To practice detecting changes in a directory over time (polling)
- To practice file classification, quarantine, and restore workflows
- To write a Makefile with multiple targets and a pre-build step

Overview

In this lab you will build a simplified antivirus daemon: a shell script that periodically checks a directory for changes, and when a change is detected, scans the directory for files it considers malicious. Malicious files are flagged, printed to the terminal, quarantined into a separate directory, and removed from the original location. You will also build a restore tool that lets a user pick quarantined files from a list, one at a time, and decide whether each was falsely flagged or genuinely malicious.

To keep the lab simple, assume `dir` contains only files, directly inside it — no subdirectories at all. You do not need to handle nested folders anywhere in this lab: not while scanning, not while quarantining, and not while restoring.

A Makefile is used to build and maintain groups of programs and files from source code with a single command. In Linux, it is one of the most frequently used tools by developers, and it will let you run your antivirus daemon, the restore tool, and the pre-build setup step with one simple command (e.g. `make`, `make restore`). You can read more about how Makefiles work here:

<https://opensource.com/article/18/8/what-how-makefile>

Part 1: Write the Antivirus Daemon

Write a script `antivirusd.sh` that will monitor a directory and quarantine malicious files.

```
antivirusd.sh dir malicious_dir interval-secs
```

Parameters:

- *dir* — the source directory being monitored (files only, no subdirectories)
- *malicious_dir* — the destination directory where flagged/quarantined files are copied
- *interval-secs* — time to wait between every check

Detecting Changes

To detect whether `dir` has changed since the last check, compare directory listings taken at two points in time:

- Before the first scan, run `ls -l dir > directory-info.last`. The command `ls -l` lists the entries directly inside `dir` along with the last modification time of every file.

- Then, every interval-secs, run `ls -l dir > directory-info.new` to capture the current state into a second file, `directory-info.new`.
- To find out whether anything changed, compare the two files with `diff` or `cmp`
- If they are identical (no change), simply wait another interval-secs and repeat.
- If they differ (a change happened), perform a scan of `dir` (see below), then copy `directory-info.new` into `directory-info.last` (e.g. `cp directory-info.new directory-info.last`), so the next comparison is against the current state.

The overall behavior of the daemon should be:

- On the very first run, there is no `directory-info.last` file yet — the script should go ahead and perform a scan immediately, then create `directory-info.last` from the current state.
- After that, every interval-secs, the script takes a new snapshot (`directory-info.new`) and compares it to `directory-info.last`.
- If nothing changed, the script simply waits another interval-secs and checks again (no scan is performed).
- If something changed, the script performs a scan of `dir`, and then updates `directory-info.last` so the next comparison is against the current state.

What Counts as a Malicious File

A file is considered malicious if it matches at least one of the following rules.

- **Flagged extension:** the file's extension matches one of the following, hardcoded exactly as written: `.exe`, `.bat`, `.vbs`, `.scr`, `.ps1`.
- **Flagged content:** the file's contents contain one of the following keywords (case-insensitive), hardcoded exactly as written: `virus`, `trojan`, `malware`, `worm`, `ransomware`.

These are the complete required lists — do not add, remove, or substitute entries. A file that matches either rule is treated as malicious, unless it has previously been marked safe (see Bonus 2 — Whitelist). Document where these lists live in your code in your README.

Action on Detection

For every file identified as malicious during a scan, the script must:

1. Print to the terminal: "<file> is malicious and it is DELETED"
2. Copy the file into `malicious_dir`, keeping its original filename
3. Delete the original file from `dir`

You can assume a given filename is never quarantined twice at the same time — once a file is flagged, it is removed from `dir`, and it will have been either restored or permanently deleted (see Part 2) before a file of the same name could be flagged again.

Part 2: Restore Tool

Write a separate script `restore.sh` that lets a user review files currently sitting in quarantine, in case a file was falsely flagged.

```
restore.sh dir malicious_dir
```

The two scripts do not interact concurrently — they should not be run at the same time.

The script repeats the following loop: List the files currently in `malicious_dir` (numbered for example), ask the user to pick one by number, and then for the picked file, prompt with three options:

- Input 1: Restore this file back into `dir` (it was a false positive)

Dr. Samia Hafez

- Input 2: Permanently delete this file from malicious_dir (it was genuinely malicious)
- Input 3: Leave this file as-is and go back to the list

Terminal Logging

Your script must log the following (adapt the exact filename/variable to whatever you use):

- "Restored <file> to <dir>." — after option 1
- "<file> permanently deleted." — after option 2
- "No malicious files to review." — if malicious_dir is empty when the tool starts

Part 3: Write a Makefile

Create a Makefile that:

1. Has a target that runs antivirusd.sh with its required arguments
2. Has a target that runs restore.sh with its required arguments
3. Has a pre-build step that creates the malicious_dir directory if it does not already exist

Part 4: Write a README file

A **README** is a text file that introduces and explains a project. It contains information that is commonly required to understand what the project is about. For guidance on what to include, see:

<https://www.makeareadme.com/>

Create a README.md that includes:

1. An overview of the code in the folder, with the folder hierarchy
2. Any prerequisites needed and how to install them on Ubuntu
3. Step-by-step instructions for running the antivirus and restore tools
4. Where in your code the required flagged-extensions and flagged-keywords lists are defined

Bonus 1: Cron Job

Instead of a script that loops forever with sleep, it is possible to run the same monitoring job on a fixed schedule using a cron job. You are required to implement the same scanning and quarantine behavior as Part 1, but as a cron job — this bonus does not include the interactive restore loop, since it requires user input. You can learn what a cron job is and how to build one here:

<https://www.freecodecamp.org/news/cron-jobs-in-linux>

Requirements:

1. Create a new antivirus-cron.sh file that will be run every 1 minute at second 23
2. Update the README.md file with:
 - A step-by-step manual to configure the cron job
 - Any prerequisites the user should take care of before following your instructions (services, installations, files, etc.)
 - What the cron expression should be to run this scan every 3rd Friday of the month at 12:31 am

Bonus 2: Whitelist

Right now, a file that gets restored through `restore.sh` will simply be flagged again on the next scan, since its extension or content hasn't changed. Extend your solution so that once a file is restored (marked as a false positive), it is remembered as safe and skipped by future scans even though it still matches a flagged extension or keyword.

- This information must persist across runs of the daemon (i.e. it should still be respected even if `antivirusd.sh` is stopped and restarted).
- Document in the README exactly how a file gets added to the whitelist, and how the daemon checks it during a scan.

Policies

- Students will work individually.
- You may talk together on the algorithms or functions being used, but are NOT allowed to look at anybody's code or to copy any part of code from any source, even from the web.
- **AI code generation is strictly not allowed.**
- **A minimum of 3 commits to your GitHub submission is required.**
- Revise the academic integrity note found on the class web page.

How to Submit

1. **Join the GitHub Classroom:**
 - You will receive a link to join the GitHub Classroom.
 - Click on the link to join the classroom.
 - After joining, select your student ID from the list, which will automatically create your personal repository.
2. **Access Your Repository:**
 - Once your repository is created, you will be redirected to it. The repository will be named with your student ID (e.g., 12345-lab2).
 - Important: Only you can access this repository to push your commits.
3. **Work on Your Lab:**
 - Download the repository to your local machine and make the required changes as specified in the lab instructions.
 - Make commits regularly as you complete different parts of the project (e.g., implementing the backup script, writing the restore script, creating the Makefile, etc.).
4. **Push Your Changes:**
 - After making a commit locally, push your changes to GitHub.
 - Each commit counts as a submission. Ensure that each commit includes all the necessary code and files to complete the assignment.
 - Make sure to follow the naming conventions (e.g., studentID-lab2 for your project folder name).
5. **Check for Automatic Grading:**
 - Every time you push a commit, GitHub Actions will automatically run the tests to grade your submission.
 - You will receive immediate feedback on your code (success or failure) along with the grade for that commit.
6. **Deadline and Late Submissions:**
 - The final date and time for submissions will be provided by your instructor.

- After the deadline, no further commits will be accepted. Ensure you submit all your work before the deadline.
- If you need to make additional changes, make sure to do so before the deadline and push the final commit.

7. **Final Submission:**

- You don't need to send anything manually. The repository you've been working on will be automatically used for grading.
- Make sure your work is final before the deadline, as no more commits will be allowed after that.